

INSTITUTO FEDERAL DO PARANÁ
JOÃO PEDRO DOS SANTOS HENRIQUE PLINTA

ÁRVORE B COM COMPRESSÃO DE DADOS

SUMÁRIO

Sumário	1
1 INTRODUÇÃO	2
2 A ÁRVORE B	2
2.1 Definição e propriedades	2
2.2 Implementação	3
2.2.1 Inserção	3
2.2.2 Busca	3
2.2.3 Remoção	3
2.3 Persistência em arquivo	4
2.4 Dois graus escolhidos	4
2.5 Métricas coletadas	4
3 ALGORITMOS DE COMPRESSÃO	5
3.1 LZW (<i>Lempel–Ziv–Welch</i>)	5
3.1.1 Compressão	5
3.1.2 Descompressão	5
3.1.3 Formato do arquivo	6
3.2 Huffman Canônico	6
3.2.1 Compressão	6
3.2.2 Descompressão	6
3.2.3 Formato do arquivo	7
4 TESTES E RESULTADOS	7
4.1 Ambiente de testes	7
4.2 Desempenho de inserção, busca e remoção	7
4.3 Estrutura da árvore	8
4.4 Memória utilizada	9
4.5 Compressão	9
5 ANÁLISE E DISCUSSÃO	10
5.1 Impacto do grau T no desempenho	10
5.2 Comparação entre LZW e Huffman	10
5.3 Integridade dos dados	11
6 CONCLUSÃO	11

1 INTRODUÇÃO

Este relatório descreve a implementação e os resultados do Trabalho 2 da disciplina de Estruturas de Dados Avançadas. O trabalho consiste em:

1. Implementar uma **Árvore B** com chaves do tipo *string*, com suporte a inserção, busca, remoção e persistência em arquivo binário;
2. Implementar dois **algoritmos de compressão** — LZW e Huffman Canônico — para comprimir o arquivo gerado pela árvore;
3. Medir e comparar o desempenho de ambas as estruturas em diferentes cenários.

A motivação é simular o ambiente real de sistemas operacionais e bancos de dados, que utilizam árvores B para indexação eficiente e precisam persistir essas estruturas em dispositivos de armazenamento. A compressão reduz o espaço necessário e o tempo de leitura/escrita em disco.

O trabalho é composto por dois programas independentes escritos em **C++17**:

Programa	Função
btree	Gerencia a árvore B interativamente via linha de comando
compress	Comprime e descomprime o arquivo da árvore

2 A ÁRVORE B

2.1 Definição e propriedades

A Árvore B com grau mínimo T satisfaz as seguintes propriedades:

- Todo nó tem no máximo $2T - 1$ chaves;
- Todo nó interno (exceto a raiz) tem no mínimo $T - 1$ chaves;
- A raiz tem no mínimo 1 chave (se a árvore não for vazia);
- Todo nó interno com k chaves tem exatamente $k + 1$ filhos;
- Todas as folhas estão no mesmo nível;
- As chaves em cada nó estão em ordem crescente.

A altura de uma Árvore B com N chaves é $O(\log_T N)$, o que garante custo logarítmico para inserção, busca e remoção.

2.2 Implementação

A implementação utiliza as seguintes estruturas em C++:

```
1 struct Node {  
2     int n; // numero de chaves  
3     bool leaf;  
4     std::vector<std::string> keys; // chaves[0..n-1]  
5     std::vector<Node*> ch; // filhos[0..n]  
6     Node(int t, bool lf) : n(0), leaf(lf),  
7         keys(2*t-1), ch(2*t, nullptr) {}  
8 };
```

As chaves são armazenadas como `std::string`, com comparação lexicográfica. O grau mínimo T é definido na linha de comando.

2.2.1 Inserção

A inserção utiliza a estratégia *proativa*: ao descer pela árvore, nós cheios ($n = 2T - 1$) são divididos (*split*) antes de continuar a descida. Isso evita que seja necessário subir novamente após a inserção na folha. A posição dentro de cada nó é encontrada com `std::lower_bound` (busca binária). A complexidade é $O(\log T \cdot \log_T N) = O(\log N)$.

2.2.2 Busca

A busca usa busca binária (`std::lower_bound`) para localizar a posição da chave dentro de cada nó. Ao encontrar a posição correta, desce para o filho correspondente ou retorna “não encontrado” em uma folha. Complexidade: $O(\log T \cdot \log_T N) = O(\log N)$.

2.2.3 Remoção

A remoção trata três casos:

1. **Chave em folha**: remoção direta com deslocamento dos vizinhos;
2. **Chave em nó interno com filho suficientemente cheio**: substitui pela chave predecessora ou sucessora e remove recursivamente;
3. **Chave em nó interno com filhos “magros”**: realiza fusão (*merge*) dos filhos e desce recursivamente.

2.3 Persistência em arquivo

O arquivo binário é escrito em **pré-ordem** (raiz, filho₀, filho₁, ...). Cada nó é serializado como:

Campo	Tipo	Descrição
leaf	int (4 B)	1 se folha, 0 se interno
n	int (4 B)	número de chaves
len _i	int (4 B)	comprimento da <i>i</i> -ésima chave
key _i	char[len _i]	bytes da <i>i</i> -ésima chave

O cabeçalho do arquivo armazena o grau T , permitindo carregar a árvore corretamente mesmo que o programa seja iniciado com um grau diferente. O uso de tamanho variável por chave evita o desperdício de bytes nulos que ocorreria com campos de tamanho fixo.

2.4 Dois graus escolhidos

Foram escolhidos dois graus bastante diferentes para observar o comportamento em dois extremos:

Propriedade	$T = 3$	$T = 50$
Mín. de chaves por nó	2	49
Máx. de chaves por nó	5	99
Mín. de filhos por nó interno	3	50
Máx. de filhos por nó interno	6	100
Fator de ramificação	baixo	alto
Altura esperada (100k chaves)	≈ 11	≈ 3

$T = 3$ representa uma árvore com nós pequenos e muitos níveis, similar a uma Árvore Binária de Busca balanceada (porém com fator de ramificação 3). $T = 50$ é o perfil típico de bancos de dados e sistemas de arquivos, onde cada nó corresponde a um bloco de disco e convém armazenar o máximo de chaves por acesso.

2.5 Métricas coletadas

Após cada operação de inserção, busca ou remoção, o programa exibe:

```
[nos visitados: 6 | tempo: 1.28e-06 s | memoria: 4480 KB]
```

Nós visitados Incrementado no início de cada chamada recursiva que acessa um nó.

Tempo Medido com `std::chrono::steady_clock` antes e depois da operação.

Memória Lida de `VmRSS` em `/proc/self/status` após a operação (memória física residente do processo).

3 ALGORITMOS DE COMPRESSÃO

3.1 LZW (*Lempel–Ziv–Welch*)

O LZW é um algoritmo de compressão baseado em dicionário sem parâmetros de configuração. O dicionário é inicializado com as 256 entradas correspondentes a cada byte possível e cresce dinamicamente ao longo da compressão.

3.1.1 Compressão

O algoritmo mantém uma *string* corrente w inicialmente vazia. Para cada byte c lido:

- Se $w + c$ está no dicionário, $w \leftarrow w + c$;
- Caso contrário, emite o código de w , adiciona $w + c$ ao dicionário e $w \leftarrow c$.

Ao final, emite o código de w .

A implementação usa códigos de **16 bits** (`uint16_t`), permitindo até 65.536 entradas. Quando o dicionário está cheio, ele é congelado (sem *reset*).

3.1.2 Descompressão

O descompressor mantém o mesmo dicionário (reconstruído localmente) e inverte o processo. O único caso especial ocorre quando o código recebido ainda não foi adicionado ao dicionário — nesse caso, a entrada é $w + w[0]$ (onde w é a última entrada processada).

3.1.3 Formato do arquivo

Campo	Tamanho	Conteúdo
Magic	4 B	LZW\0
Tamanho original	8 B	uint64_t LE
Número de códigos	8 B	uint64_t LE
Códigos	$N \times 2$ B	uint16_t LE cada

3.2 Huffman Canônico

O algoritmo de Huffman atribui códigos de comprimento variável a cada símbolo (byte), com códigos mais curtos para símbolos mais frequentes. A variante **canônica** garante que, dados apenas os comprimentos dos códigos, os códigos podem ser reconstruídos deterministicamente — eliminando a necessidade de serializar a árvore inteira no arquivo.

3.2.1 Compressão

1. Contar a frequência de cada byte no arquivo de entrada;
2. Construir a árvore de Huffman com uma fila de prioridade mínima;
3. Coletar os comprimentos dos códigos para cada um dos 256 símbolos possíveis;
4. Gerar os códigos canônicos: ordenar símbolos por (comprimento, valor) e atribuir códigos inteiros em ordem crescente, deslocando à esquerda a cada mudança de comprimento;
5. Codificar os dados bit a bit (MSB primeiro dentro de cada byte).

3.2.2 Descompressão

O descompressor lê os 256 comprimentos do cabeçalho, reconstrói os códigos canônicos e constrói uma **trie binária** estática em vetor. Cada nó da trie armazena dois índices de filho (bit 0 e bit 1) e o símbolo decodificado (−1 se nó interno). A decodificação percorre a trie bit a bit — ao atingir uma folha, emite o símbolo e retorna à raiz — eliminando a busca em mapa de *strings* da abordagem ingênua e reduzindo o custo de $O(k \cdot \log n)$ para $O(k)$ por símbolo (onde k é o comprimento do código).

3.2.3 Formato do arquivo

Campo	Tamanho	Conteúdo
Magic	4 B	HUF\0
Tamanho original	8 B	uint64_t LE
Padding	1 B	bits de preenchimento no último byte
Comprimentos	256 B	1 byte por símbolo (0 = ausente)
Dados	variável	fluxo de bits codificados

O cabeçalho tem tamanho fixo de **269 bytes**, o que explica a menor eficiência em arquivos pequenos.

4 TESTES E RESULTADOS

4.1 Ambiente de testes

Componente	Especificação
Sistema operacional	Linux 7.0.0
Compilador	g++, flags -O2 -std=c++17
Chaves ($N \leq 10.000$)	<i>strings</i> aleatórias de 4–12 chars (letras + dígitos)
Chaves ($N > 10.000$)	sequenciais embaralhadas (k000000000–k000999999)
Semente	42 (reprodutível)

4.2 Desempenho de inserção, busca e remoção

Para cada combinação de N e T , foram medidos os nós visitados e o tempo de todas as operações (inserções, buscas e remoções), com média calculada sobre o conjunto completo.

Tabela 1 – Desempenho médio por operação (média de insert + read + delete)

N	$T = 3$		$T = 50$		Redução em nós
	Nós visit.	Tempo (μs)	Nós visit.	Tempo (μs)	
100	2,91	0,79	1,18	0,55	–59%
10.000	6,39	0,77	2,31	0,91	–64%
100.000	8,08	1,05	2,93	0,85	–64%
1.000.000	9,93	1,20	3,54	1,28	–64%

O crescimento dos nós visitados segue a razão logarítmica esperada: ao aumentar N de 100 para 1.000.000 (fator 10.000 $\times$), $T = 3$ passa de 2,91 para 9,93 nós ($\approx 3,4\times$) e $T = 50$ passa de 1,18 para 3,54 ($\approx 3\times$). A redução relativa de nós visitados se mantém estável em torno de -64% para $N \geq 10.000$.

Ambos os graus usam busca binária dentro dos nós ($O(\log T)$ comparações por nó). O total de comparações de chave é $O(\log T \cdot \log_T N) = O(\log N)$, independentemente de T . Na prática, os tempos são próximos: para $N = 1.000.000$, $T = 3$ leva 1,20 μs e $T = 50$ leva 1,28 μs , pois $T = 50$ visita menos nós mas faz mais comparações por nó ($\lceil \log_2 99 \rceil = 7$ vs $\lceil \log_2 5 \rceil = 3$).

4.3 Estrutura da árvore

Tabela 2 – Estrutura da árvore após inserção de N chaves

N	$T = 3$			$T = 50$		
	Altura	Total nós	Fill	Altura	Total nós	Fill
100	3	31	58,1%	1	1	90,9%
10.000	7	3.108	64,3%	3	145	69,6%
100.000	9	31.210	64,1%	3	1.436	70,3%
1.000.000	10	312.680	64,0%	4	14.575	69,3%

Com $N = 1.000.000$, $T = 50$ tem apenas 4 níveis e 14.575 nós, contra 10 níveis e 312.680 nós para $T = 3$ — uma redução de $\approx 21\times$ no número de nós. O fill factor (fração de slots de chave ocupados) se estabiliza em $\approx 64\%$ para $T = 3$ e $\approx 70\%$ para $T = 50$ a partir de $N = 10.000$.

As médias da Tabela 1 incluem busca e remoção. O comportamento de busca e remoção é consistente com a inserção: $T = 50$ visita em média $\approx 64\%$ menos nós em todos os tamanhos. Buscas por chaves inexistentes percorrem a árvore até uma folha, visitando tantos nós quanto a altura. A remoção visita nós adicionais nos casos em que é necessário realizar *merge* ou empréstimo de chaves entre nós irmãos.

4.4 Memória utilizada

Tabela 3 – Memória residente (VmRSS) ao final da inserção de N chaves

N	$T = 3$ (KB)	$T = 50$ (KB)	Redução
100	4.232	4.232	0%
10.000	5.148	4.788	−7%
100.000	13.492	9.928	−26%
1.000.000	97.012	62.068	−36%

Com $T = 50$, a memória é **36% menor** para 1.000.000 de chaves (≈ 95 MB vs ≈ 61 MB). Embora cada nó com $T = 50$ seja individualmente maior (até 99 slots de chave), o número total de nós é $\approx 21 \times$ menor, resultando em menor uso total de memória.

4.5 Compressão

As árvores foram comprimidas com ambos os algoritmos para $T = 3$ e $T = 50$.

Tabela 4 – Taxa de compressão — $T = 3$

N	Original	LZW		Huffman	
		Comprimido	Taxa	Comprimido	Taxa
100	1,3 KB	1,7 KB	$0,81 \times$	1,0 KB	$1,29 \times$
10.000	141,6 KB	88,3 KB	$1,60 \times$	82,2 KB	$1,72 \times$
100.000	1.610,9 KB	384,8 KB	$4,19 \times$	637,8 KB	$2,52 \times$
1.000.000	15,7 MB	4,4 MB	$3,60 \times$	6,5 MB	$2,41 \times$

Tabela 5 – Taxa de compressão — $T = 50$

N	Original	LZW		Huffman	
		Comprimido	Taxa	Comprimido	Taxa
100	1,1 KB	1,5 KB	$0,72 \times$	0,94 KB	$1,17 \times$
10.000	118,4 KB	84,0 KB	$1,41 \times$	73,4 KB	$1,61 \times$
100.000	1.378,3 KB	353,3 KB	$3,90 \times$	548,2 KB	$2,51 \times$
1.000.000	13,5 MB	3,9 MB	$3,46 \times$	5,7 MB	$2,38 \times$

Tabela 6 – Tempo de compressão e descompressão ($T = 3$, descompressão Huffman com trie)

N	LZW		Huffman	
	Compressão	Descompressão	Compressão	Descompressão
100	0,60 ms	0,11 ms	0,14 ms	0,16 ms
10.000	17,5 ms	3,53 ms	3,64 ms	2,09 ms
100.000	142 ms	10,0 ms	16,8 ms	22,3 ms
1.000.000	1.535 ms	73,8 ms	186 ms	210 ms

5 ANÁLISE E DISCUSSÃO

5.1 Impacto do grau T no desempenho

O resultado mais significativo é a diferença no número de nós visitados entre $T = 3$ e $T = 50$. Com 100.000 chaves, $T = 3$ visita em média **8,08 nós** por inserção, enquanto $T = 50$ visita apenas **2,93**. Isso reflete diretamente a diferença de altura:

$$h_{T=3}(N) = \left\lceil \log_3 \frac{N+1}{2} \right\rceil \quad h_{T=50}(N) = \left\lceil \log_{50} \frac{N+1}{2} \right\rceil$$

Para $N = 100.000$: $h_{T=3} \approx 11$ e $h_{T=50} \approx 3$.

Em sistemas de banco de dados reais, onde cada nó representa um bloco de disco (tipicamente 4–16 KB), a redução de ≈ 8 para ≈ 3 acessos a disco por operação representa uma melhoria expressiva. Com latência de disco de ≈ 5 ms por acesso, a economia seria de ≈ 25 ms por operação.

No contexto em memória principal, ambos os graus usam busca binária dentro dos nós. Como o número total de comparações de chave é $O(\log N)$ independentemente de T , as diferenças de tempo são modestas. Para $N = 1.000.000$, $T = 50$ é ligeiramente mais lento (1,28 μ s vs 1,20 μ s): embora visite menos nós, cada nó com até 99 chaves exige $\lceil \log_2 99 \rceil = 7$ comparações contra $\lceil \log_2 5 \rceil = 3$ para $T = 3$, compensando parte do ganho em profundidade.

5.2 Comparação entre LZW e Huffman

O comportamento dos dois algoritmos varia conforme o tamanho do arquivo e o tipo de chave:

- Para $N = 100$ e $N = 10.000$ (chaves aleatórias): Huffman supera o LZW em taxa (1,29×/1,72× vs 0,81×/1,60× para $T = 3$). O LZW chega a **expandir** o arquivo para

$N = 100$ ($0,81\times$), pois o *overhead* do cabeçalho supera o ganho em um arquivo muito pequeno;

- Para $N = 100.000$ e $N = 1.000.000$ (chaves sequenciais): LZW supera expressivamente o Huffman ($4,19\times$ vs $2,52\times$ para $T = 3$, $N = 100.000$). A alta regularidade das chaves sequenciais (k000000000, k000000001, ...) cria padrões de bytes repetitivos que o dicionário LZW explora muito eficientemente;
- **Queda do LZW em 1.000.000:** a taxa cai de $4,19\times$ ($N = 100.000$) para $3,60\times$ ($N = 1.000.000$). Com o arquivo maior, o dicionário de 65.536 entradas satura antes de processar todo o conteúdo e novos padrões não podem mais ser registrados, reduzindo a eficiência;
- **Tempo:** o Huffman é $\approx 10\times$ mais rápido na compressão para todos os tamanhos. A descompressão LZW é rápida (≈ 74 ms para 1M chaves); a descompressão Huffman leva ≈ 210 ms via trie binária estática, que percorre bit a bit com custo $O(k)$ por símbolo (onde k é o comprimento do código), sendo $\approx 7\times$ mais rápida que a abordagem por mapeamento de *strings*.

5.3 Integridade dos dados

Em todos os testes, os arquivos descomprimidos foram verificados com `cmp` e são **bit a bit idênticos** aos originais. Isso confirma a corretude dos dois algoritmos de compressão e descompressão.

6 CONCLUSÃO

O trabalho demonstrou que a Árvore B é uma estrutura de indexação eficiente para até 1.000.000 de chaves, com operações na casa dos microssegundos e crescimento logarítmico no número de nós acessados.

A escolha do grau T tem impacto direto na estrutura: com $T = 50$ e $N = 1.000.000$, a árvore tem apenas 4 níveis e 14.575 nós, contra 10 níveis e 312.680 nós para $T = 3$ ($21\times$ menos nós). Em memória principal, com busca binária dentro dos nós, o total de comparações é $O(\log N)$ para qualquer T . O ganho em nós visitados (-64%) não se traduz em ganho proporcional de tempo: para $N = 1.000.000$, $T = 50$ é marginalmente mais lento ($1,28$ vs $1,20 \mu s$), pois compensa a menor profundidade com mais comparações por nó. Em sistemas orientados a disco, onde cada acesso a nó representa um bloco físico, T grande seria claramente superior.

Em relação à compressão, o desempenho depende fortemente do tamanho do arquivo e da regularidade das chaves. Para arquivos pequenos com chaves aleatórias ($N \leq 10.000$),

o **Huffman** é superior em taxa e é $\approx 10\times$ mais rápido. Para arquivos grandes com chaves sequenciais ($N \geq 100.000$), o **LZW** supera o Huffman expressivamente em taxa ($4,19\times$ vs $2,52\times$ para $N = 100.000$), pois o dicionário acumula os padrões altamente repetitivos das chaves sequenciais. Porém, o LZW perde eficiência ao saturar seu dicionário de 65.536 entradas ($3,60\times$ para $N = 1.000.000$ vs $4,19\times$ para $N = 100.000$). O Huffman mantém tempo de compressão baixo e uso de memória estável em todos os cenários; sua descompressão via trie binária (≈ 210 ms para 1M chaves) é mais lenta que o LZW (≈ 74 ms) mas opera em tempo $O(k)$ por símbolo.

Como trabalho futuro, seria interessante avaliar: (1) LZW com códigos de largura variável para superar o limite de 65.536 entradas; (2) serialização em largura (*level-order*) ao invés de pré-ordem, pois nós do mesmo nível têm conteúdo similar e poderiam ser melhor comprimidos juntos.